# CSE 209 — Computer Architecture

## Implementation of the MIPS Control Unit (Mapping Control to Hardware)

*Lesson notes · Exam approach · TF practice with solutions · Quick notes (Rifat Sir's portion)*

### How to Read These Notes

| | |
|---|---|
| **EXAM FOCUS** | Exam importance, where it appears in the TF collection |
| **MEMORIZE** | Rules, tables and formulas to memorize |
| **TRAP** | Common mistakes and exam traps |
| **EXAM-READY ANSWER** | Model wording you can write in the exam |
| **NOTE** | Extra explanation / clarification |
| **Grey box, monospace** | Code, bit layouts and hand-drawn datapath sketches |
| **Centered grey box** | Formula to remember / final numerical result |
| **Purple heading / ✓ green** | Practice questions / their solutions |

### Contents

**Exam Map — What the TF Papers Ask** — control-unit questions and how much they matter

**Lesson 29 — Combinational Control: Truth Tables, Gates and PLA** — ALU control equations, main control equations, PLA

**Lesson 30 — Multicycle Datapath and the Control FSM** — prerequisite datapath, signals, 10 states, traces, CPI

**Lesson 31 — FSM Logic: Output/Next-State Equations, ROM and PLA Sizes** — equations, ROM words, ROM vs PLA size

**Lesson 32 — Sequencers, Microprogramming, Hardwired vs Microprogrammed** — AddrCtl, dispatch ROMs, microprogram, exam answers

**Quick Notes — Control Unit** — merged revision notes + key diagrams

> **NOTE** Page numbers refer to the **Implementation of the MIPS Control Unit** PDF (32 pages, Appendix D "Mapping Control to Hardware"). The appendix assumes the **multicycle datapath**, which is not in the PDF; Lesson 30 gives the short version you need (from the textbook, marked). Exception handling in the multicycle design (asked in 2012-13) also follows the textbook and is marked.

# Exam Map — What the TF Papers Ask

| Session (paper) | Control-unit questions |
|---|---|
| **2012-13** (CSE 305) | Q3(a) **Present and explain a microprogram** for the multicycle control unit (17) · Q3(b) modify the multicycle implementation for **undefined-instruction and overflow exceptions** (8) · Q3(c) in which step should **overflow** be detected so the destination register is not changed? (5) · Q3(d) can **PCWriteCond be replaced by PCSource[0]**? (5) |
| Older paper (TF p. 84) | Q3(c) **properties of hardwired and microprogrammed** control units (6) |
| Related (Ch 4 P1) | "Explain how the ALU control bits are set from ALUOp and the function code" (TF p. 55, Q2(b)) — Lesson 29 gives the gate-level version |

**EXAM FOCUS** **No control-unit question appears in the 2019-20 to 2023-24 papers.** Treat this part as insurance: learn the **FSM (10 states and their signals)**, the **microprogram**, **ROM vs PLA sizes**, and the **hardwired vs microprogrammed** comparison. Those cover every question that has been asked.

# Lesson 29 — Combinational Control: Truth Tables, Gates and PLA

*PDF pages 2–7 (§D.1–D.2)*

## 1. Two Kinds of Control (p. 2)

- **Combinational control** has **no state**: outputs depend only on the current inputs. Examples: the **ALU control** unit, and the **main control of the single-cycle** datapath.
- **Sequential control** has a **state register** and handles a **multicycle** design: outputs depend on the current state, and the next state depends on the state and the opcode. It can be described as a **finite-state diagram** or as a **microprogram** (each microinstruction = one state).

## 2. Worked Example: ALU Control → Gates (pp. 3–5)

Start from the Chapter 4 truth table (inputs ALUOp1, ALUOp0 and funct F5–F0; output the 4-bit Operation):

| ALUOp 1 | ALUOp 0 | Funct F5…F0 | Operation |
|---|---|---|---|
| 0 | 0 | XXXXXX | 0010 (add — lw/sw) |
| 0 | 1 | XXXXXX | 0110 (sub — beq) |
| 1 | 0 | XX0000 | 0010 (add) |
| 1 | X | XX0010 | 0110 (sub) |
| 1 | 0 | XX0100 | 0000 (AND) |
| 1 | 0 | XX0101 | 0001 (OR) |
| 1 | X | XX1010 | 0111 (slt) |

**Step 1 — one truth table per output bit (only the rows where it is 1), simplified with don't-cares:**

| Output | Rows that make it 1 |
|---|---|
| Operation3 | none — always 0 (only NOR uses it, not in this subset) |
| Operation2 | ALUOp0 = 1 (beq) **or** ALUOp1 = 1 with F1 = 1 (sub, slt) |
| Operation1 | ALUOp1 = 0 (lw, sw, beq) **or** F2 = 0 (add, sub, slt) |
| Operation0 | ALUOp1 = 1 with F0 = 1 (or) **or** ALUOp1 = 1 with F3 = 1 (slt) |

**Step 2 — logic equations:**

$$\textbf{Operation3} = 0$$
$$\textbf{Operation2} = \textbf{ALUOp0} + \textbf{ALUOp1} \cdot \textbf{F1}$$
$$\textbf{Operation1} = \textbf{ALUOp1}' + \textbf{F2}'$$
$$\textbf{Operation0} = \textbf{ALUOp1} \cdot (\textbf{F3} + \textbf{F0})$$

Only **F3–F0** are needed (F5, F4 are always don't-care). ALUOp = 11 never occurs, so it is a don't-care that lets "10" be written as "1X" and "01" as "X1" — that is why Operation2 needs only ALUOp0.

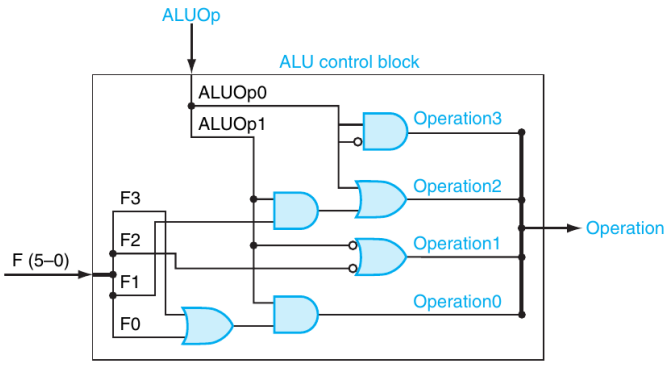



*Figure 29.1 — ALU control block built from the equations (p. 5)*

**MEMORIZE** **Check by substitution:** lw/sw (ALUOp 00) → 0, 0, 1, 0 = 0010 ✓ · beq (01) → 0, 1, 1, 0 = 0110 ✓ · slt (10, F = 101010) → 0, 1, 1, 1 = 0111 ✓ (Operation2 = F1 = 1, Operation1 = F2' = 1, Operation0 = F3 = 1).

## 3. Main Control → Gates or a PLA (pp. 6–7)

**Inputs:** opcode Op5–Op0. **Decode each instruction with one AND term:**

| Instruction | Opcode | Product term |
|---|---|---|
| R-format | 000000 | Op5' Op4' Op3' Op2' Op1' Op0' |
| lw | 100011 | Op5 Op4' Op3' Op2' Op1 Op0 |
| sw | 101011 | Op5 Op4' Op3 Op2' Op1 Op0 |
| beq | 000100 | Op5' Op4' Op3' Op2 Op1' Op0' |

**Then each output is an OR of the instructions that assert it** (single-cycle table):

| | | |
|---|---|---|
| RegDst = R-format | ALUSrc = lw + sw | MemtoReg = lw |
| RegWrite = R-format + lw | MemRead = lw | MemWrite = sw |
| Branch = beq | ALUOp1 = R-format | ALUOp0 = beq |

A **PLA (programmable logic array)** implements exactly this two-level structure: an **AND plane** that forms the product terms (here 4) from the inputs and their complements, followed by an **OR plane** that sums them into the outputs. Any function can be computed with two levels of logic, so a PLA is one of the most common ways to implement control.

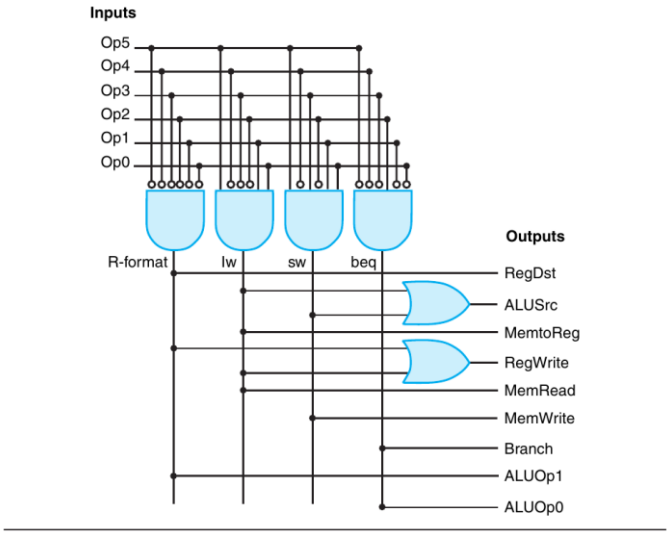



*Figure 29.2 — PLA for the single-cycle main control: 4 AND terms, 9 OR outputs (p. 7)*

## Exam Approach

1. Write the truth table (only the 1-rows), marking don't-cares.
2. One equation per output = OR of the AND terms of its rows; simplify with don't-cares.
3. Draw gates (or a PLA: AND plane → OR plane) and label inputs/outputs.
4. **PLA size ∝ (#inputs × #product terms) + (#outputs × #product terms).**

## Practice — Lesson 29

*(a) Adapted — TF p. 55 Q2(b) at gate level. (b)–(c) Adapted — textbook exercises D.2/D.3.*

**(a)** Derive the logic equations for the four ALU control bits from ALUOp and the funct field, and verify them for add and or.

**(b)** How many product terms does the single-cycle main-control PLA need if **addi** (opcode 001000, writes rt with ALU result, ALUSrc = 1, ALUOp = 00) is added? Which output equations change?

**(c)** What is the size of the single-cycle main-control PLA (4 instructions) in PLA cells, using the size formula?

### ✓ Solution

**(a)** Equations as in §2. add (ALUOp 10, F = 100000): Operation2 = 0 + 1·0 = 0; Operation1 = 0 + 1 = 1; Operation0 = 1·(0 + 0) = 0 → **0010** ✓. or (F = 100101): Operation2 = 0 + 1·0 = 0; Operation1 = 0 + 0 = 0 (F2 = 1); Operation0 = 1·(0 + 1) = 1 → **0001** ✓.

**(b)** One more product term (addi = Op5' Op4' Op3 Op2' Op1' Op0') → **5 product terms**. Changed outputs: **ALUSrc = lw + sw + addi**, **RegWrite = R-format + lw + addi**; RegDst = R-format still gives 0 (rt) for addi, MemtoReg = lw gives 0 (ALU result), ALUOp = 00.

**(c)** Inputs 6, outputs 9, product terms 4 → (6 × 4) + (9 × 4) = **60 cells**.

# Lesson 30 — Multicycle Datapath and the Control FSM

*PDF pages 7–8 (Figure D.3.1) · multicycle datapath and signal meanings follow the textbook*

**EXAM FOCUS** Every later question (equations, ROM, microprogram, exceptions, PCWriteCond) is built on the **10-state FSM**. Memorize the states and the signals asserted in each.

## 1. The Multicycle Datapath in One Page (textbook)

Each instruction is broken into steps of **one clock cycle each**, so a functional unit can be **reused** in different cycles:

- **One memory** for instructions and data; **one ALU** (also computes PC + 4 and the branch target); no separate adders.
- Extra registers hold values between cycles: **IR** (instruction register), **MDR** (memory data register), **A**, **B** (register-file outputs), **ALUOut**.
- Each instruction takes only as many cycles as it needs → **CPI varies** by instruction class.

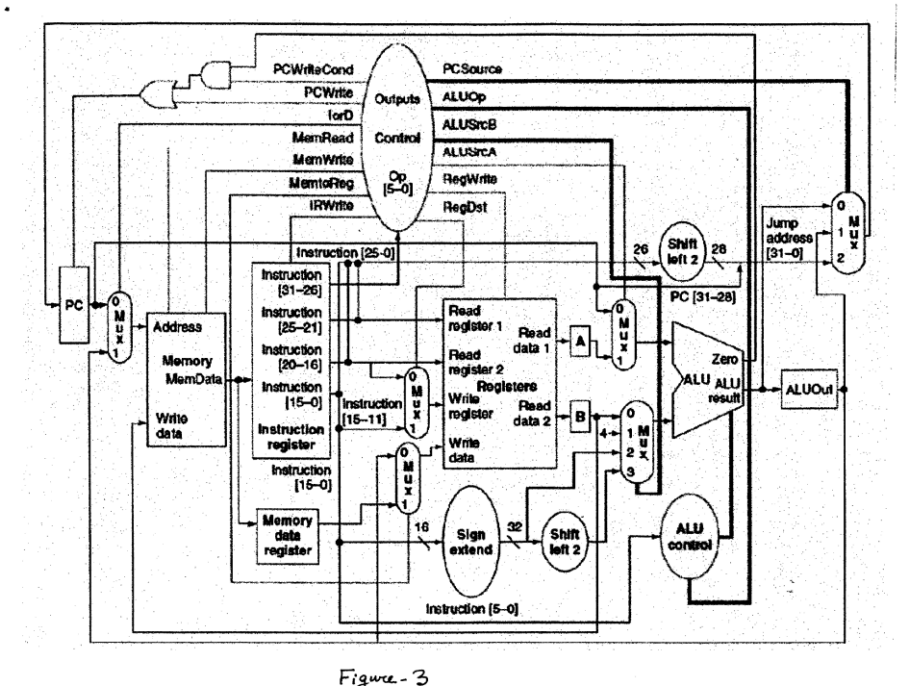



Figure-3

*Figure 30.1 — Multicycle datapath with control (as given in TF 2012-13, Figure 3)*

| Signal | Value | Effect |
|---|---|---|
| **IorD** | 0 / 1 | Memory address = **PC** (fetch) / **ALUOut** (data access) |
| **MemRead / MemWrite** | 1 | Read / write the memory |
| **IRWrite** | 1 | Write the memory output into **IR** |
| **RegDst** | 0 / 1 | Destination = **rt** / **rd** |
| **MemtoReg** | 0 / 1 | Register write data = **ALUOut** / **MDR** |
| **RegWrite** | 1 | Write the register file |
| **ALUSrcA** | 0 / 1 | First ALU input = **PC** / **A** |
| **ALUSrcB** | 00 / 01 / 10 / 11 | Second ALU input = **B** / **4** / **sign-extended imm** / **sign-extended imm << 2** |
| **ALUOp** | 00 / 01 / 10 | ALU **adds** / **subtracts** / uses **funct** |
| **PCSource** | 00 / 01 / 10 | New PC = **ALU result** (PC + 4) / **ALUOut** (branch target) / **jump address** |
| **PCWrite** | 1 | Write PC unconditionally |
| **PCWriteCond** | 1 | Write PC **only if Zero = 1** (beq) |

**PC write enable = PCWrite + (PCWriteCond · Zero)**

## 2. The Five Steps

| Step | Instruction(s) | Register-transfer action |
|---|---|---|
| 1 Fetch | all | IR ← Mem[PC]; PC ← PC + 4 |
| 2 Decode | all | A ← Reg[rs]; B ← Reg[rt]; ALUOut ← PC + (sign-ext(imm) << 2) (branch target computed early) |
| 3 Execute | lw/sw | ALUOut ← A + sign-ext(imm) |
| | R-type | ALUOut ← A op B |
| | beq | if (A == B) PC ← ALUOut |
| | j | PC ← {PC[31:28], IR[25:0], 00} |
| 4 Memory / R completion | lw | MDR ← Mem[ALUOut] |
| | sw | Mem[ALUOut] ← B |
| | R-type | Reg[rd] ← ALUOut |
| 5 Write-back | lw | Reg[rt] ← MDR |

## 3. The Finite-State Machine (p. 8)

| State | Name | Signals asserted (others 0 / don't care) → next state |
|---|---|---|
| **0** | Instruction fetch | MemRead, IorD = 0, IRWrite, ALUSrcA = 0, ALUSrcB = 01, ALUOp = 00, PCWrite, PCSource = 00 → **1** |
| **1** | Decode / register fetch | ALUSrcA = 0, ALUSrcB = 11, ALUOp = 00 → **2** (lw/sw), **6** (R-type), **8** (beq), **9** (j) |
| **2** | Memory address computation | ALUSrcA = 1, ALUSrcB = 10, ALUOp = 00 → **3** (lw), **5** (sw) |
| **3** | Memory access (lw) | MemRead, IorD = 1 → **4** |
| **4** | Write-back (lw) | RegDst = 0, RegWrite, MemtoReg = 1 → **0** |
| **5** | Memory access (sw) | MemWrite, IorD = 1 → **0** |
| **6** | Execution (R-type) | ALUSrcA = 1, ALUSrcB = 00, ALUOp = 10 → **7** |
| **7** | R-type completion | RegDst = 1, RegWrite, MemtoReg = 0 → **0** |
| **8** | Branch completion | ALUSrcA = 1, ALUSrcB = 00, ALUOp = 01, PCWriteCond, PCSource = 01 → **0** |
| **9** | Jump completion | PCWrite, PCSource = 10 → **0** |

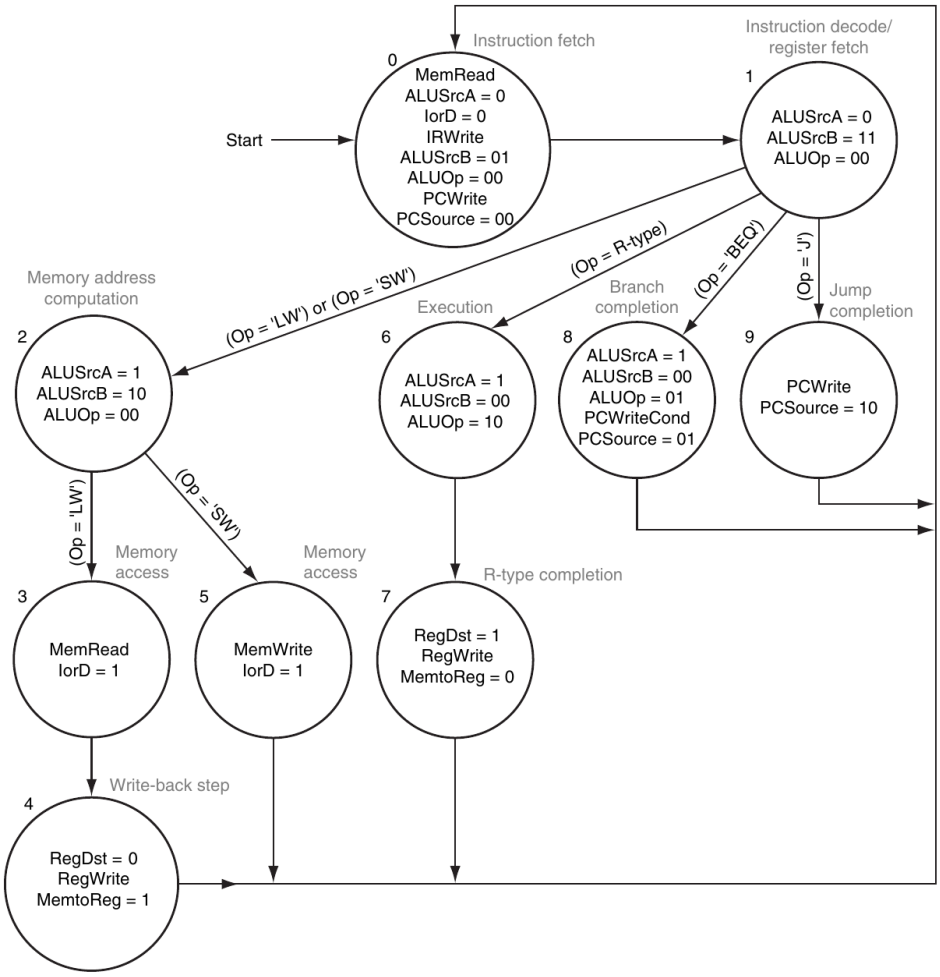



*Figure 30.2 — Finite-state diagram for multicycle control (p. 8)*

**Cycles per instruction:** lw **5** (0-1-2-3-4) · sw **4** (0-1-2-5) · R-type **4** (0-1-6-7) · beq **3** (0-1-8) · j **3** (0-1-9).

## 4. Worked Trace: `add $t2, $t0, $t1`

| State | What happens |
|---|---|
| 0 | IR ← Mem[PC], PC ← PC + 4 (ALU adds PC + 4) |
| 1 | A ← $t0, B ← $t1; ALUOut ← branch target (unused); opcode 000000 → next state 6 |
| 6 | ALU computes A + B (ALUOp 10, funct = add) → ALUOut |
| 7 | $t2 ← ALUOut (RegDst = 1, MemtoReg = 0, RegWrite) → back to 0 |

**4 cycles.**

**TRAP**

- In state 1 the ALU computes the branch target for **every** instruction — it is harmless and saves a cycle for beq.
- beq uses **PCSource = 01** (ALUOut, computed in state 1), not the ALU result of state 8 (which is A − B, used only for Zero).
- In states 0 and 9, PCWrite is asserted; in state 8 it is **PCWriteCond**, not PCWrite.

## Practice — Lesson 30

*Constructed (multicycle trace and CPI).*

**(a)** List the states visited and the signals asserted for **lw $t0, 8($s0)**. How many cycles does it take?

**(b)** A program has 25 % lw, 10 % sw, 52 % R-type, 11 % beq and 2 % j. What is the CPI of the multicycle implementation?

**(c)** Why does state 1 compute PC + (sign-ext(imm) << 2) even when the instruction is not a branch?

### ✓ Solution

**(a)** 0 (fetch: MemRead, IorD = 0, IRWrite, ALUSrcA = 0, ALUSrcB = 01, ALUOp = 00, PCWrite, PCSource = 00) → 1 (decode: ALUSrcA = 0, ALUSrcB = 11, ALUOp = 00) → 2 (address: ALUSrcA = 1, ALUSrcB = 10, ALUOp = 00) → 3 (MemRead, IorD = 1) → 4 (RegDst = 0, RegWrite, MemtoReg = 1). **5 cycles.**

**(b)**

$$\text{CPI} = 0.25 \times 5 + 0.10 \times 4 + 0.52 \times 4 + 0.11 \times 3 + 0.02 \times 3 = 1.25 + 0.40 + 2.08 + 0.33 + 0.06 = \mathbf{4.12}$$

**(c)** The instruction is not decoded until the end of state 1, and the ALU is idle in that state anyway. Computing the target "optimistically" costs nothing, and if the instruction is a beq the target is ready in ALUOut, so beq finishes in state 8 (3 cycles instead of 4).

# Lesson 31 — FSM Logic: Output/Next-State Equations, ROM and PLA Sizes

*PDF pages 7–20 (§D.3)*

## 1. The Control Unit Structure (pp. 7–9)

10 states → **4 state bits S3–S0** (state i encoded as binary i) held in a **state register**. The **control logic** is combinational with:

- **Inputs (10):** opcode Op5–Op0 + state S3–S0.
- **Outputs (20): 16 datapath control signals** + **4 next-state bits NS3–NS0** (written into the state register at the clock edge).

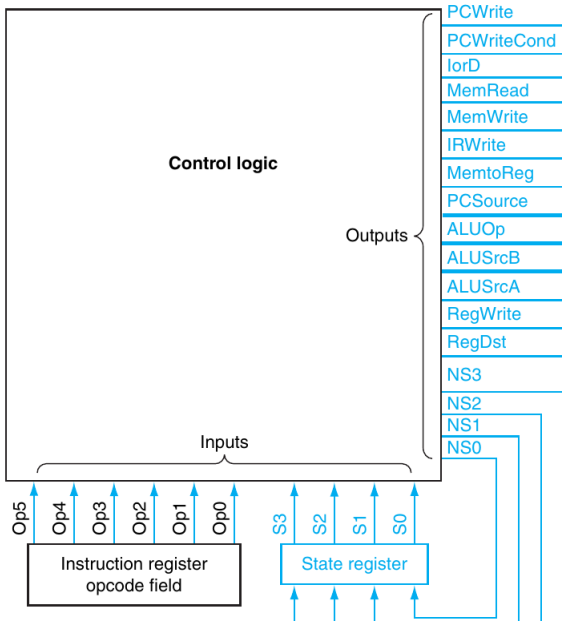



*Figure 31.1 — Control unit = control logic + state register (p. 9)*

## 2. Output Equations — Depend on the State Only (p. 10)

| Output | Asserted in | Output | Asserted in |
|---|---|---|---|
| PCWrite | state0 + state9 | ALUOp1 | state6 |
| PCWriteCond | state8 | ALUOp0 | state8 |
| IorD | state3 + state5 | ALUSrcB1 | state1 + state2 |
| MemRead | state0 + state3 | ALUSrcB0 | state0 + state1 |
| MemWrite | state5 | ALUSrcA | state2 + state6 + state8 |
| IRWrite | state0 | RegWrite | state4 + state7 |
| MemtoReg | state4 | RegDst | state7 |
| PCSource1 | state9 | PCSource0 | state8 |

"stateN" means the AND of the state bits for N, e.g. **state6 = S3' · S2 · S1 · S0'** (0110).

## 3. Next-State Equations — Depend on State and Opcode (pp. 10–12)

| Next state | Condition |
|---|---|
| NextState1 | state0 |
| NextState2 | state1 · (Op = lw + Op = sw) |
| NextState3 | state2 · (Op = lw) |
| NextState4 | state3 |
| NextState5 | state2 · (Op = sw) |
| NextState6 | state1 · (Op = R-type) |
| NextState7 | state6 |
| NextState8 | state1 · (Op = beq) |
| NextState9 | state1 · (Op = jmp) |
| NextState0 | state4 + state5 + state7 + state8 + state9 |

**Each NS bit = OR of the NextState terms whose number has that bit = 1:**

**NS0** = NextState1 + NextState3 + NextState5 + NextState7 + NextState9
**NS1** = NextState2 + NextState3 + NextState6 + NextState7
**NS2** = NextState4 + NextState5 + NextState6 + NextState7
**NS3** = NextState8 + NextState9

**Worked example (p. 11) — NS0 expanded:**

NextState1 = S3' S2' S1' S0'
NextState3 = S3' S2' S1 S0' · Op5 Op4' Op3' Op2' Op1 Op0 (state 2, lw)
NextState5 = S3' S2' S1 S0' · Op5 Op4' Op3 Op2' Op1 Op0 (state 2, sw)
NextState7 = S3' S2 S1 S0' (state 6)
NextState9 = S3' S2' S1' S0 · Op5' Op4' Op3' Op2' Op1 Op0' (state 1, jmp)

NS0 is the OR of these five product terms.

## 4. ROM Implementation (pp. 14–18)

Use the 10 inputs as the **address** and store the 20 outputs as the **word**:

**Single ROM = $2^{10}$ words × 20 bits = 20 Kbits**

Wasteful: the 16 datapath outputs depend only on the 4 state bits, so they are repeated 64 times. **Split it into two ROMs:**

**Two ROMs = $2^4$ × 16 + $2^{10}$ × 4 = 256 + 4096 = 4352 bits ≈ 4.3 Kbits**
(about 1/5 of 20 Kbits)

| State (S3–S0) | Datapath control bits 19–4 (PCWrite … RegDst) | Active signals |
|---|---|---|
| 0000 | 1001 0100 0000 1000 | PCWrite, MemRead, IRWrite, ALUSrcB0 |
| 0001 | 0000 0000 0001 1000 | ALUSrcB1, ALUSrcB0 |
| 0010 | 0000 0000 0001 0100 | ALUSrcB1, ALUSrcA |
| 0011 | 0011 0000 0000 0000 | IorD, MemRead |
| 0100 | 0000 0010 0000 0010 | MemtoReg, RegWrite |
| 0101 | 0010 1000 0000 0000 | IorD, MemWrite |
| 0110 | 0000 0000 0100 0100 | ALUOp1, ALUSrcA |
| 0111 | 0000 0000 0000 0011 | RegWrite, RegDst |
| 1000 | 0100 0000 1010 0100 | PCWriteCond, PCSource0, ALUOp0, ALUSrcA |
| 1001 | 1000 0001 0000 0000 | PCWrite, PCSource1 |

Bit order (19 → 4): PCWrite, PCWriteCond, IorD, MemRead, MemWrite, IRWrite, MemtoReg, PCSource1, PCSource0, ALUOp1, ALUOp0, ALUSrcB1, ALUSrcB0, ALUSrcA, RegWrite, RegDst. The low 4 bits (NS) come from the next-state table: e.g. state 0001 with lw (100011) → **0010**.

**Example (p. 16):** for which addresses is PCWrite = 1? States 0 and 9 → all addresses **XXXXXX0000** and **XXXXXX1001** (opcode bits are don't-cares). **Example (p. 18):** word at address 100011 0001 (lw, state 1) = 0000000000011000 **0010**.

## 5. PLA Implementation (pp. 19–20)

A PLA stores only the **product terms that are needed** (one copy each, with don't-cares). This control function needs **17 product terms**: 10 that depend only on the state + 7 that combine opcode and state.

**PLA size ∝ (#inputs × #terms) + (#outputs × #terms) = (10 × 17) + (20 × 17) = 510 cells**

**Split into two PLAs:** outputs PLA (4 inputs, 10 terms, 16 outputs) = (4 × 10) + (16 × 10) = **200**; next-state PLA (10 inputs, 7 terms, 4 outputs) = (10 × 7) + (4 × 7) = **98** → **298 cells** (≈ 55 % of one PLA).

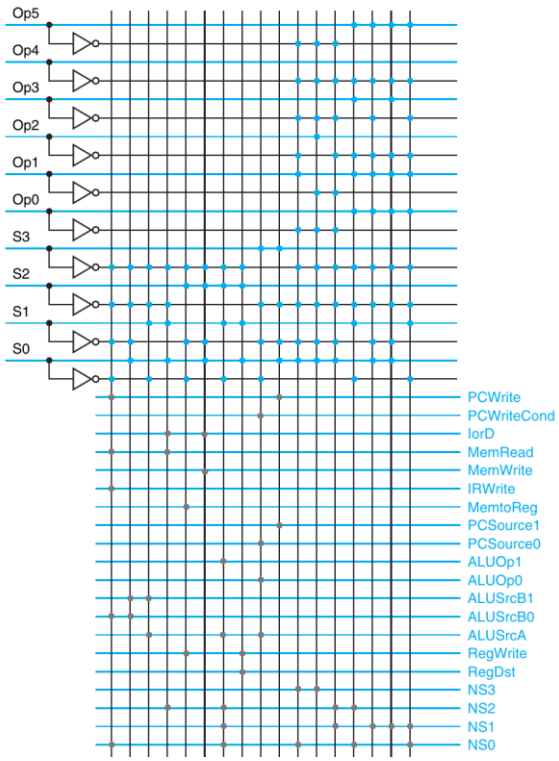



*Figure 31.2 — PLA implementing the multicycle control (p. 20)*

**MEMORIZE** **Size ladder:** single ROM 20 Kbits → two ROMs 4.3 Kbits → single PLA 510 cells → two PLAs 298 cells. **PLAs win unless the control function is very dense.**

**Simplification ideas (pp. 12–13, 26):** PCWriteCond, PCSource0 and ALUOp0 are all asserted **only in state 8** → they can be replaced by **one signal**. **State assignment:** RegWrite is active in states 4 and 7; renumber them 8 and 9 and RegWrite = S3 alone.

## Practice — Lesson 31

*(a) Actual — 2012-13 Q3(d). (b) Constructed (sizes). (c) Constructed (equations).*

**(a)** Your friend claims that the PCWriteCond signal can be safely replaced by PCSource[0]. Do you agree? Justify.

**(b)** Suppose the instruction set grows so that the FSM has **20 states** and **18 datapath control signals** (opcode still 6 bits). Find the size of (i) a single control ROM, (ii) the split two-ROM version.

**(c)** Write the equations for NS3 and NS2 in terms of states and opcode tests.

### ✓ Solution

**(a) Agree, for this FSM.** PCSource0 = 1 only in state 8 (PCSource = 01) and PCWriteCond = 1 only in state 8 — their truth tables are identical (p. 13 notes they can be one signal). The PC enable becomes PCWrite + PCSource0 · Zero, which gives exactly the same behaviour. **Conditions:** PCSource0 must be driven **0** (not X) in all other states, otherwise a Zero = 1 from the ALU could write the PC by accident; and if the design is extended so that PCSource0 = 1 is used without a conditional write (e.g. a new PCSource = 11 input for exceptions), the two signals are no longer equivalent unless PCWrite is also asserted there.

**(b)** State bits = $\lceil \log_2 20 \rceil$ = **5**; inputs = 6 + 5 = 11; outputs = 18 + 5 = 23.

| |
|---|
| (i) $2^{11} \times 23 = 2048 \times 23 =$ **47,104 bits** |
| (ii) $2^5 \times 18 + 2^{11} \times 5 = 576 + 10{,}240 =$ **10,816 bits** |

**(c)**

| |
|---|
| NS3 = NextState8 + NextState9 = state1 · (Op = beq) + state1 · (Op = jmp) |
| NS2 = NextState4 + NextState5 + NextState6 + NextState7 = state3 + state2 · (Op = sw) + state1 · (Op = R-type) + state6 |

# Lesson 32 — Sequencers, Microprogramming, Hardwired vs Microprogrammed

*PDF pages 21–31 (§D.4–D.6) · microprogram layout and exception states follow the textbook*

**EXAM FOCUS** **2012-13 Q3(a):** present and explain a microprogram (17 marks). **Older paper:** properties of hardwired and microprogrammed control. **2012-13 Q3(b)–(c):** exceptions in the multicycle design.

## 1. Why a Sequencer? (pp. 21–22)

In the two-ROM design, **4096 of 4368 bits (94 %)** encode the next-state function. Most states simply go to the **next numbered state** (e.g. 3 → 4). So replace the next-state logic with a **counter**: an **adder** increments the state, and extra **address-control (AddrCtl)** bits choose how the next state is formed.

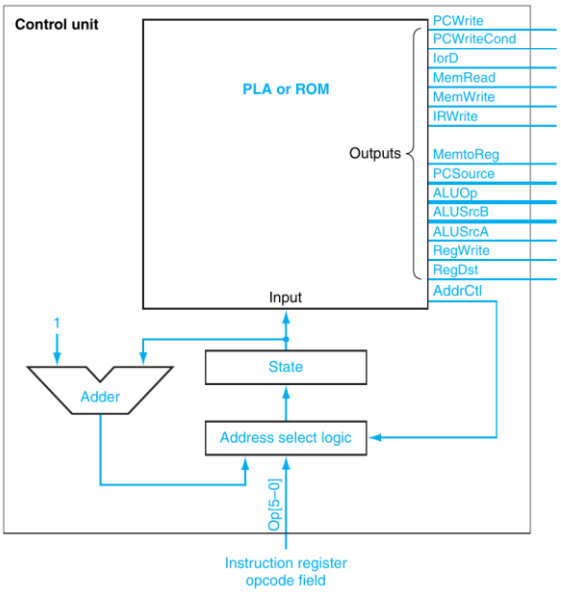



*Figure 32.1 — Control unit with an explicit counter (sequencer) (p. 22)*

## 2. Address Select Logic (pp. 23–25)

| AddrCtl | Action |
|---|---|
| 0 | Set state to **0** (fetch the next instruction) |
| 1 | **Dispatch with ROM 1** (branch on opcode after state 1) |
| 2 | **Dispatch with ROM 2** (branch on opcode after state 2) |
| 3 | Use the **incremented** state |

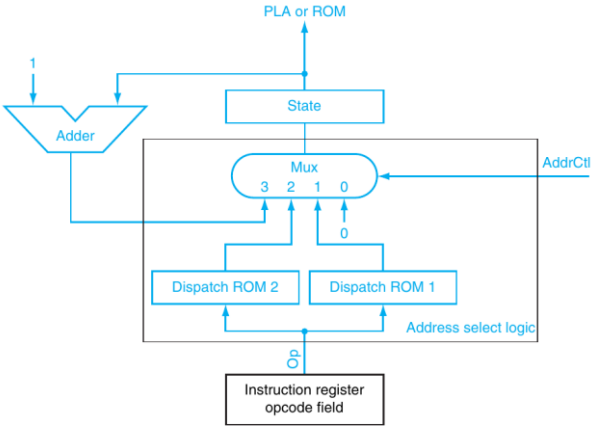



*Figure 32.2 — Address select logic: adder, two dispatch ROMs and a 4-input mux (p. 24)*

| Dispatch ROM 1 (Op) | Value | Dispatch ROM 2 (Op) | Value |
|---|---|---|---|
| 000000 R-format | 0110 (6) | 100011 lw | 0011 (3) |
| 000010 jmp | 1001 (9) | 101011 sw | 0101 (5) |
| 000100 beq | 1000 (8) | | |
| 100011 lw | 0010 (2) | | |
| 101011 sw | 0010 (2) | | |

| State | Action | AddrCtl | State | Action | AddrCtl |
|---|---|---|---|---|---|
| 0 | Incremented | 3 | 5 | To 0 | 0 |
| 1 | Dispatch ROM 1 | 1 | 6 | Incremented | 3 |
| 2 | Dispatch ROM 2 | 2 | 7 | To 0 | 0 |
| 3 | Incremented | 3 | 8 | To 0 | 0 |
| 4 | To 0 | 0 | 9 | To 0 | 0 |

**Storage:** 10 control words × 18 bits (16 datapath + 2 AddrCtl) = **180 bits** + 2 dispatch ROMs × 64 × 4 = **512 bits** → **692 bits** (vs 4.3 Kbits).

## 3. Control as Microcode (pp. 26–28)

The sequencer looks like a small computer: the ROM/PLA is a **microcode memory** holding **microinstructions**, the state register is the **microprogram counter**. Writing the control as a symbolic program is **microprogramming**.

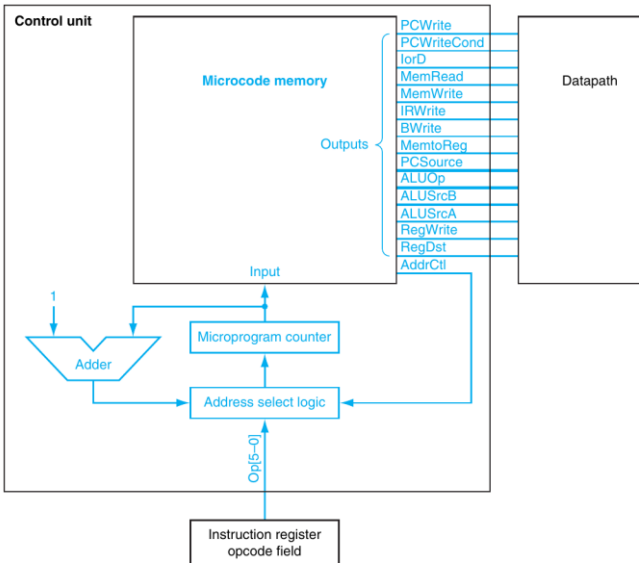



*Figure 32.3 — The control unit viewed as microcode (p. 27)*

## Microinstruction fields (p. 28)

| Field | Values | Signals |
|---|---|---|
| ALU control | Add · Subt · Func code | ALUOp = 00 · 01 · 10 |
| SRC1 | PC · A | ALUSrcA = 0 · 1 |
| SRC2 | B · 4 · Extend · Extshft | ALUSrcB = 00 · 01 · 10 · 11 |
| Register control | Read · Write ALU · Write MDR | Read rs, rt into A, B · RegWrite, RegDst = 1, MemtoReg = 0 · RegWrite, RegDst = 0, MemtoReg = 1 |
| Memory | Read PC · Read ALU · Write ALU | MemRead, IorD = 0, IRWrite · MemRead, IorD = 1 · MemWrite, IorD = 1 |
| PC write control | ALU · ALUOut-cond · Jump address | PCSource = 00, PCWrite · PCSource = 01, PCWriteCond · PCSource = 10, PCWrite |
| Sequencing | Seq · Fetch · Dispatch 1 · Dispatch 2 | AddrCtl = 11 · 00 · 01 · 10 |

## ★ The microprogram (answer to 2012-13 Q3(a))

| Label | ALU control | SRC1 | SRC2 | Register control | Memory | PCWrite control | Sequencing |
|---|---|---|---|---|---|---|---|
| **Fetch** | Add | PC | 4 | | Read PC | ALU | Seq |
| | Add | PC | Extshft | Read | | | Dispatch 1 |
| **Mem1** | Add | A | Extend | | | | Dispatch 2 |
| **LW2** | | | | | Read ALU | | Seq |
| | | | | Write MDR | | | Fetch |
| **SW2** | | | | | Write ALU | | Fetch |
| **Rformat1** | Func code | A | B | | | | Seq |
| | | | | Write ALU | | | Fetch |
| **BEQ1** | Subt | A | B | | | ALUOut-cond | Fetch |
| **JUMP1** | | | | | | Jump address | Fetch |

Dispatch table 1: R-format → Rformat1, jmp → JUMP1, beq → BEQ1, lw/sw → Mem1. Dispatch table 2: lw → LW2, sw → SW2.

**EXAM-READY ANSWER** **Explanation line by line:** *Fetch* reads the instruction at PC into IR and writes PC + 4 into the PC (ALU adds PC and 4); the next microinstruction computes the branch target PC + (sign-ext << 2) into ALUOut, reads rs and rt into A and B, and dispatches on the opcode. *Mem1* computes the memory address A + sign-ext(imm) and dispatches again (lw or sw). *LW2* reads memory at ALUOut into MDR, then writes MDR into rt and returns to Fetch. *SW2* writes B into memory at ALUOut. *Rformat1* performs the funct-specified operation on A and B, then writes ALUOut into rd. *BEQ1* subtracts A − B and writes ALUOut (the target) into PC if Zero. *JUMP1* writes the jump address into PC. Each microinstruction corresponds to one state of the FSM (0–9).

## 4. Reducing the Control Store (pp. 30–31)

- **Encoding:** if at most one of k control lines is active at a time, store them in a $\lceil \log_2 k \rceil$-bit field and decode (e.g. 8 lines → 3 bits, saving 5 bits per word), at the cost of a decoder.
- **Format fields:** different microinstruction formats (e.g. memory vs ALU) with a format/opcode field → narrower words, but some operations need two microinstructions.
- **Horizontal microcode** = wide, minimally encoded, few microinstructions, maximum parallel control. **Vertical microcode** = narrow, highly encoded, more microinstructions.

## 5. Hardwired vs Microprogrammed Control ★

| | Hardwired (FSM in logic / PLA) | Microprogrammed (microcode) |
|---|---|---|
| Control signals from | Combinational logic of the FSM (gates, PLA) | Microinstructions read from a **control memory** (ROM) |
| Next state | Encoded next-state logic | **Microprogram counter** + sequencer (increment, dispatch, fetch) |
| Speed | **Faster** — no control-memory access | Slower — a control-store read every cycle |
| Flexibility | Hard to change; redesign the logic | **Easy to modify / extend** — rewrite the microprogram |
| Design effort | Grows quickly with many instructions/states | Systematic, like writing a program — good for **large, complex (CISC) ISAs** |
| Hardware cost | Small for simple ISAs (PLA is very efficient) | Control store can be large for complex ISAs |
| Typical use | Simple, fast **RISC** processors | Complex instruction sets (e.g. older x86, VAX) |

**Big picture (p. 31):** whether control is drawn as an FSM or written as a microprogram, the hardware is similar — each state/microinstruction asserts outputs and says how to pick the next state. An **explicit sequencer** pays off when there are **many states and long sequential runs**; **PLAs beat ROMs** unless the function is very dense (ROMs suit a separate control memory).

## 6. Exceptions in the Multicycle Design (textbook — 2012-13 Q3(b), (c))

**Hardware additions:**

1. **EPC** register (32 bits) with **EPCWrite**; it is written with **PC − 4** (the PC was already incremented in fetch), computed by the ALU: ALUSrcA = 0, ALUSrcB = 01, ALUOp = 01.
2. **Cause** register with **CauseWrite**, and a mux selected by **IntCause** (0 = undefined instruction, 1 = arithmetic overflow).
3. A fourth PC-source input: **PCSource = 11** → **8000 0180$_{hex}$** (handler address).
4. ALU **Overflow** output fed to the control unit.

**Two new states:**

| State | Signals |
|---|---|
| **10 — Undefined instruction** (from state 1 when the opcode matches none) | IntCause = 0, CauseWrite, ALUSrcA = 0, ALUSrcB = 01, ALUOp = 01, EPCWrite, PCWrite, PCSource = 11 → 0 |
| **11 — Overflow** | IntCause = 1, CauseWrite, ALUSrcA = 0, ALUSrcB = 01, ALUOp = 01, EPCWrite, PCWrite, PCSource = 11 → 0 |

**Where to detect overflow (Q3(c)):** the ALU produces Overflow while computing A op B in the **execute step (state 6)**. Use it at the end of state 6 to go to state 11 **instead of state 7**. State 7 is where RegWrite is asserted, so the offending instruction never writes its destination register. Detecting it later (after or during state 7) would let the wrong result overwrite rd.

> NOTE Some textbook versions draw the overflow arc **out of state 7**; in that case the write must be blocked by gating RegWrite with NOT Overflow. Detecting in state 6 (as above) avoids the write without extra gating — say which one you use.

## Practice — Lesson 32

*(a) Actual — 2012-13 Q3(a). (b) Actual — older paper (TF p. 84) Q3(c). (c) Constructed (sizes). (d) Actual — 2012-13 Q3(c).*

**(a)** Present and explain a microprogram for the control unit of the multicycle implementation.

**(b)** Briefly describe the properties of hard-wired and microprogrammed control unit design.

**(c)** With an explicit sequencer, the design gets 2 more states (for addi: compute A + sign-ext, then write rt). What is the total control storage (control words + dispatch ROMs)?

**(d)** In which step should the overflow exception be detected so that the offending instruction does not change its destination register? Justify.

### ✓ Solution

**(a)** The microprogram table and line-by-line explanation in §3.

**(b)** The comparison table in §5. Short form: hardwired control generates signals from fixed logic (gates/PLA) implementing the FSM — fast and compact but difficult to change; microprogrammed control stores the control signals as microinstructions in a control memory sequenced by a microprogram counter — flexible, systematic and easy to extend, but slower and needing a control store.

**(c)** 12 states (still 4 state bits, so dispatch entries stay 4 bits wide): 12 × 18 = **216 bits** + 2 × 64 × 4 = **512 bits** → **728 bits**. (addi also needs a dispatch-ROM-1 entry, which fits in the existing 64 entries.)

**(d)** In the **execute step (state 6)**, when the ALU computes the result. Branch to the overflow state 11 instead of state 7, so the register write in state 7 never happens and rd keeps its old value (precise exception); EPC gets PC − 4 so the handler knows the offending instruction.

# Quick Notes — Control Unit

*Revision points for Lessons 29–32.*

## L29 · Combinational Control

| | |
|---|---|
| Operation3 = 0 | Operation2 = ALUOp0 + ALUOp1·F1 |
| Operation1 = ALUOp1' + F2' | Operation0 = ALUOp1·(F3 + F0) |

- Only F3–F0 used; ALUOp = 11 is a don't-care.
- Main control: decode R/lw/sw/beq with 4 AND terms; RegDst = R · ALUSrc = lw + sw · MemtoReg = lw · RegWrite = R + lw · MemRead = lw · MemWrite = sw · Branch = beq · ALUOp1 = R · ALUOp0 = beq.
- PLA = AND plane → OR plane; size ∝ (inputs + outputs) × product terms.

## L30 · Multicycle FSM

- One memory, one ALU, registers IR, MDR, A, B, ALUOut. PC enable = PCWrite + PCWriteCond·Zero.
- ALUSrcB: 00 B · 01 4 · 10 imm · 11 imm << 2. PCSource: 00 ALU · 01 ALUOut · 10 jump.
- States: 0 fetch · 1 decode · 2 address · 3 lw mem · 4 lw WB · 5 sw mem · 6 R exec · 7 R done · 8 beq · 9 j.
- Cycles: lw 5, sw 4, R 4, beq 3, j 3. Mix 25/10/52/11/2 % → CPI 4.12.

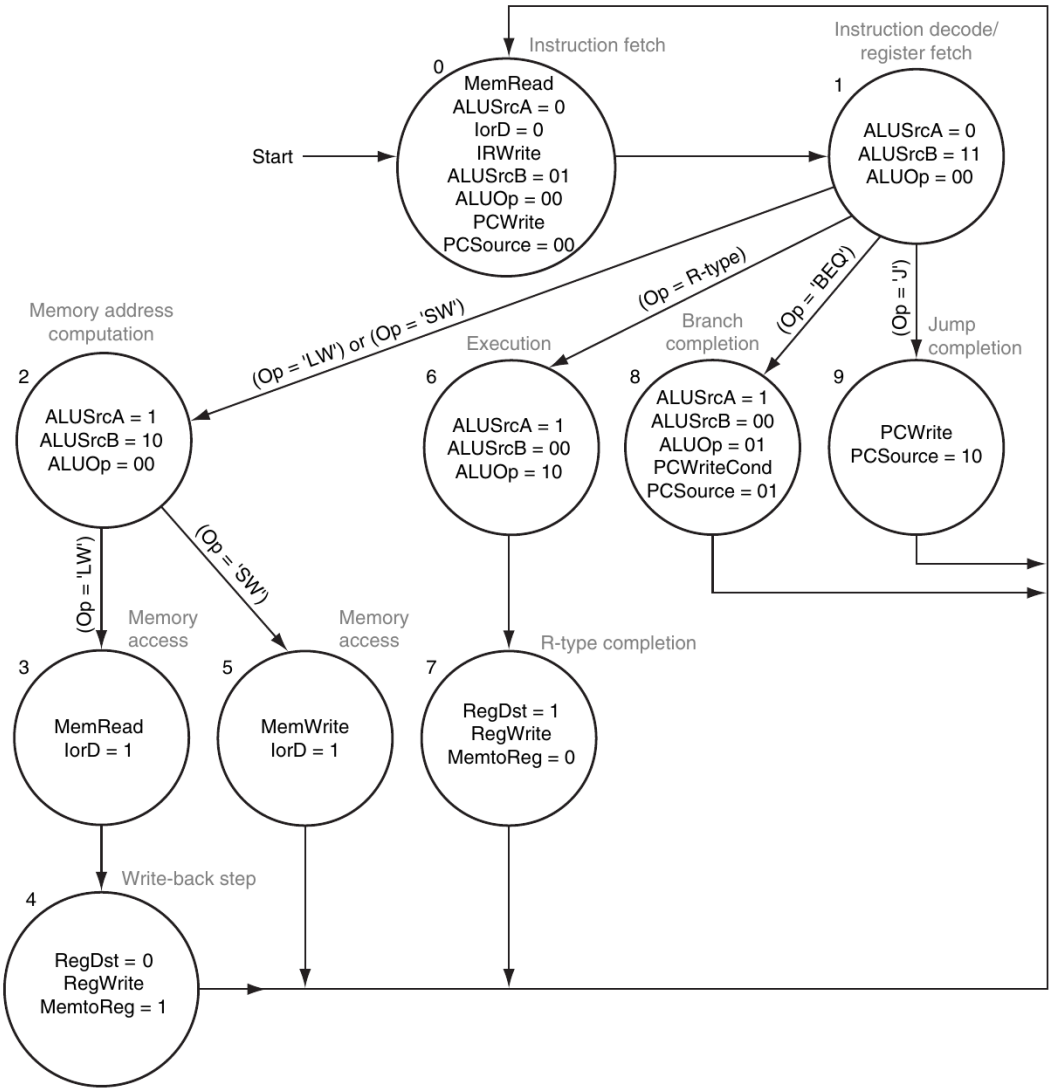
## L31 · Equations and Sizes

- 4 state bits; inputs 10 (Op + S), outputs 20 (16 control + 4 NS).
- NS0 = NextState1 + NextState3 + NextState5 + NextState7 + NextState9; NS3 = state1·(beq + jmp); NS2 = state3 + state2·sw + state1·R + state6.
- ROM $2^{10}$ × 20 = **20 Kbits**; split $2^4$ × 16 + $2^{10}$ × 4 = **4.3 Kbits**.
- PLA 17 terms: (10 + 20) × 17 = **510**; split 200 + 98 = **298**.
- PCWriteCond = PCSource0 = ALUOp0 (all only in state 8) → can be one signal. State assignment simplifies logic.

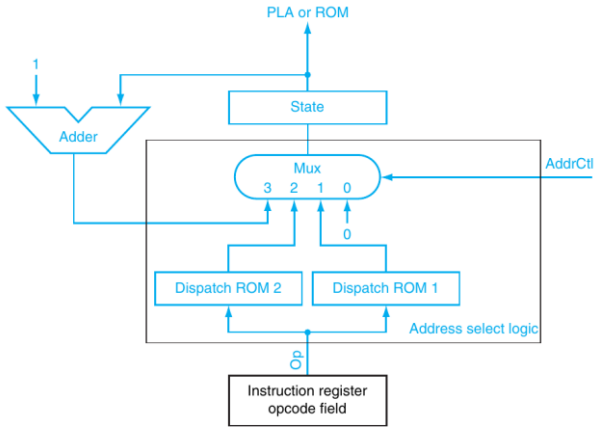
## L32 · Sequencer and Microcode

- AddrCtl: 0 fetch · 1 dispatch ROM 1 · 2 dispatch ROM 2 · 3 increment. Storage 10 × 18 + 2 × 64 × 4 = **692 bits**.
- Microprogram labels: Fetch (2 lines), Mem1, LW2 (2), SW2, Rformat1 (2), BEQ1, JUMP1.
- Encoding / format fields; horizontal (wide) vs vertical (narrow) microcode.
- Hardwired: fast, rigid, RISC. Microprogrammed: flexible, systematic, slower, complex ISAs.
- Exceptions: EPC (= PC − 4), Cause, IntCause, PCSource 11 → $8000\ 0180_{hex}$; states 10 (undefined), 11 (overflow); detect overflow in state 6 so state 7 never writes rd.

## Key Diagrams to Reproduce




*Finite-state diagram for multicycle control (p. 8) — same as Fig. 30.2*




*Address select logic with dispatch ROMs (p. 24) — same as Fig. 32.2*